

LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

“Ekspansi Produk Keuangan (Inheritance)”

Disusun Oleh:

Sofian Arba'i (2511533029)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. Tahapan Praktikum

Berikut adalah beberapa tahapan untuk praktikum Pemrograman Berorientasi Objek pekan 4 dengan judul "Ekspansi Produk Keuangan (Inheritance)".

1.1 Tujuan

Tujuan dari praktikum pekan 4 dengan judul "Ekspansi Produk Keuangan (Inheritance)" sebagai berikut.

1. Memahami prinsip Code Reusability melalui Pewarisan (Inheritance).
2. Membedakan Superclass dan Subclass
3. Menggunakan keyword `extends` untuk mewariskan sifat, serta `super()` untuk memanggil konstruktor induk.
4. Menerapkan Upcasting dan Downcasting pada koleksi objek

1.2 Alat

Alat yang dibutuhkan untuk melakukan praktikum pekan 4 sebagai berikut:

1. Laptop/PC
2. JDK dan Eclipse

1.3 Pemahaman Teori

Seiring berkembangnya sistem perbankan, bank tidak hanya memiliki satu jenis rekening. Ada "Rekening Tabungan" yang mana akan mendapat bunga bulanan dan "Rekening Giro" yang memiliki fasilitas pinjaman atau overdraft. Jika kita membuat class baru dari nol untuk masing masing entitas tersebut, kita akan melanggar prinsip DRY (Don't Repeat Yourself) karena menduplikasi banyak kode logika PIN dan mutasi.

- Konsep Pewarisan (Inheritance): Menurut Deitel dan Deitel (2017), pewarisan adalah bentuk software *reuse* di mana class baru diciptakan dengan

menggunakan atribut dan behavior dari class yang sudah ada, lalu menambahkan kapabilitas baru. Hubungan ini dikenal sebagai relasi "Is-A" contohnya Tabungan is-a Rekening.

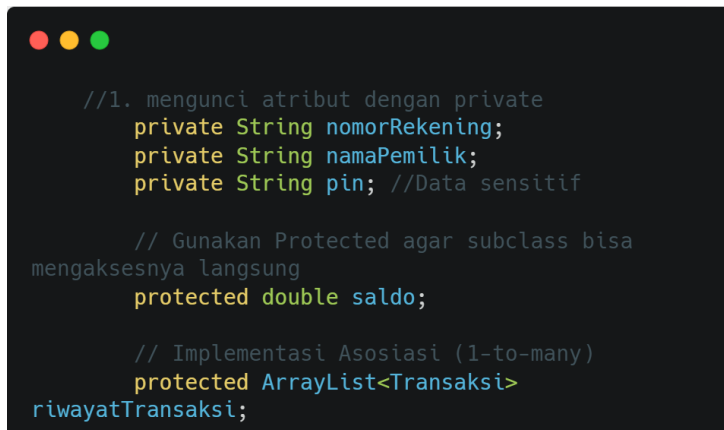
- Penggunaan Keyword extends dan super: Di Java, subclass mewarisi kelas induk menggunakan kata kunci extends. Sementara itu, kata kunci super berfungsi sebagai jembatan langsung ke kelas induk.
 - `super()` digunakan di dalam konstruktor kelas anak untuk mengeksekusi inisialisasi konstruktor induk. Pemanggilan ini wajib berada di baris paling pertama di dalam konstruktor anak (Oracle, 2023).
 - `super.namaMethod()` digunakan jika kelas anak perlu memanggil metode asli milik induk yang mungkin telah dimodifikasi (di-override).
- Pentingnya protected: Agar kelas anak (Tabungan, Giro) bisa memanipulasi saldo dan mencatat riwayatTransaksi tanpa harus membuka akses tersebut ke publik, atribut tersebut harus menggunakan modifier protected.
- Upcasting dan Downcasting: Karena adanya relasi Is-A, Java mengizinkan konversi tipe referensi antar hierarki kelas
- Upcasting (Implisit): Menyimpan objek kelas anak ke dalam variabel referensi kelas induk contohnya `Rekening akun = new RekeningTabungan(...)`. Proses ini otomatis dan aman. Fungsinya sangat krusial agar kita bisa menyimpan berbagai jenis produk rekening ke dalam satu koleksi yang seragam seperti ArrayList tipe Rekening.
- Downcasting (Eksplisit): Memaksa referensi kelas induk kembali menjadi kelas anak spesifik contoh `RekeningTabungan tab = (RekeningTabungan) akun`. Proses ini rawan memicu error `ClassCastException` saat program berjalan jika objek aslinya ternyata bukan Tabungan. Oleh karena itu, downcasting wajib didahului oleh pengecekan tipe menggunakan operator `instanceof`.

1.4 Langkah-Langkah

Pada praktikum ini, saya akan menjadikan Class Rekening.java menjadi kelas induk (Superclass), lalu membuat produk spesifik berupa Tabungan dan Giro.

1.4.1 Langkah 1: Memastikan Akses Modifier di Rekening.java

Membuka file Rekening.java dan memastikan atribut saldo dan riwayatTransaksi menggunakan hak akses protected, sementara pin dan nomorRekening tetap private. Berikut merupakan perubahan kode program langkah 1.



```
//1. mengunci atribut dengan private
private String nomorRekening;
private String namaPemilik;
private String pin; //Data sensitif

// Gunakan Protected agar subclass bisa
mengaksesnya langsung
protected double saldo;

// Implementasi Asosiasi (1-to-many)
protected ArrayList<Transaksi>
riwayatTransaksi;
```

Perubahan yang terjadi pada kode program pekan 4 dengan pekan 3 pada langkah 1 yaitu perubahan pada saldo yang sebelumnya private menjadi protected. Perubahan tersebut menjadikan subclass RekeningTabungan bisa langsung menulis saldo += nominalBunga; di method tambahBungaAkhirBulan() tanpa getter/setter. Kalau tetap private, baris itu akan error karena saldo hanya bisa diakses di dalam Rekening.

1.4.2 Langkah 2: Membuat Subclass Rekening Tabungan

Membuat class baru bernama RekeningTabungan.java yaitu sebuah produk Tabungan yang memiliki fitur tambahan yaitu Suku Bunga Bulanan. Berikut adalah kode program untuk pembuatan class RekeningTabungan.java.

```

package pekan4;

public class RekeningTabungan extends Rekening {
    // Atribut spesifik yang hanya dimiliki oleh
    tabungan
    private double sukuBunga;

    //constructor Subclass
    public RekeningTabungan(String nomor, String
nama, double saldoAwal, String pinAwal, double
sukuBunga) {
        // Super () memanggil constructor kelas
        untuk (rekening). Wajib berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        //Menghitung Bunga
        //Mengapa bisa mengakses saldo secara
        langsung dari class RekeningTabungan?
        double nominalBunga = saldo *
(sukuBunga/100);
        saldo += nominalBunga;

        //Mencatat riwayat transaksi
        String idTrx = "TRX-B-" +
System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx,
"Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "%
berhasil ditambahkan Rp" + nominalBunga);
    }
}

```

Penjelasan kode program diatas dibagi menjadi beberapa bagian sebagai berikut:

1. Deklarasi Package dan Atribut

Terdapat pada package pekan4 dan Atribut khusus Tabungan yaitu sukuBunga, tidak ada di Rekening. Bersifat private, jadi hanya bisa dipakai di dalam class RekeningTabungan. Serta extends Rekening membuatnya mewarisi semua atribut dan method induk (setor, tarik, mutasi, otentikasi, dan lainnya).

2. Constructor

Constructor dijalankan satu kali saat objek dibuat dengan `new RekeningTabungan(...)`. Constructor ini menerima lima parameter: nomor, nama, saldo awal, PIN awal, dan suku bunga. Empat parameter pertama sebenarnya adalah data milik Rekening (induk), sehingga baris pertama `super(nomor, nama, saldoAwal, pinAwal)`; meneruskannya ke constructor induk agar nomor rekening, nama pemilik, saldo, PIN, daftar riwayat, dan status keamanan terisi lebih dulu. Pemanggilan `super()` wajib berada di baris paling pertama, karena objek induk harus terbentuk sebelum bagian anak dibuat. Setelah itu, baris `this.sukuBunga = sukuBunga`; mengisi atribut yang hanya dimiliki Tabungan. Kata `this` dipakai untuk membedakan atribut milik objek dari parameter yang namanya sama.

3. Method `tambahBungaAkhirBulan()`

Method ini adalah method biasa yang menambahkan bunga bulanan ke saldo. Pertama, method menghitung nominal bunga dengan rumus `saldo * (sukuBunga / 100)`; Nilai ini langsung ditambahkan ke saldo lewat `saldo += nominalBunga`; Saldo bisa diakses langsung dari subclass karena di Rekening atributnya bersifat `protected`. Kedua, method mencatat transaksi dengan membuat ID unik dari `System.currentTimeMillis()` berawalan "TRX-B-", lalu menambahkan objek Transaksi bertipe "Bunga" ke `riwayatTransaksi`, yang juga `protected`

1.4.3 Langkah 3: Membuat Subclass `RekeningGiro`

Membuat class baru bernama `RekeningGiro.java` yaitu sebuah produk yang digunakan oleh pebisnis dan memiliki sebuah fitur tambahan yaitu Batas Overdraft (Limit Pinjaman). Berikut adalah kode program dari pembuatan class `RekeningGiro`.

```

package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama,
double saldoAwal, String pinAwal, double
batasOverdraft) {
        // Memanggil Inisialisasi dasar dari
superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}

```

Penjelasan kode program diatas dibagi menjadi beberapa bagian sebagai berikut.

1. Deklarasi Package, Class dan Atribut

Deklarasi class (subclass). package pekan4; menempatkannya satu package dengan Rekening, dan extends Rekening membuatnya mewarisi semua atribut dan method induk (setor, tarik, mutasi, otentikasi, dan lainnya). Atribut tambahan private double batasOverdraft; yaitu Atribut khusus Giro berupa batas pinjaman (limit overdraft). Bersifat private, jadi hanya bisa dipakai di dalam class ini.

2. Constructor

Constructor RekeningGiro menerima lima parameter: nomor, nama, saldo awal, PIN awal, dan batas overdraft. Seperti pada RekeningTabungan, empat parameter pertama adalah data milik Rekening, sehingga baris super(nomor, nama, saldoAwal, pinAwal); meneruskannya ke constructor induk agar bagian dasar objek terbentuk lebih dulu. Pemanggilan ini wajib berada di baris pertama. Setelah itu,

`this.batasOverdraft = batasOverdraft;` mengisi atribut yang hanya dimiliki Giro, dengan `this` membedakan atribut dari parameter yang namanya sama.

3. Getter

Memiliki return type (`double`), dan hanya mengembalikan nilai atribut tanpa memprosesnya. Karena `batasOverdraft` bersifat `private`, getter ini adalah satu-satunya cara bagi class lain (misalnya `Main`) untuk membaca nilainya.

2. Latihan atau Tugas

Pada praktikum pekan 4 Sekarang sistem telah memiliki berbagai bentuk rekening. Silahkan memperbarui antarmuka CLI di `Main.java` agar mendukung produk-produk baru ini secara interaktif.

1. Tugas 1: Pembuatan Multi-Produk (40%)

Pada tugas 1, program diminta untuk modifikasi case 1 dengan ketentuan sebagai berikut:

- Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
- Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek `RekeningTabungan`.
- Jika user memilih Giro, minta input `batasOverdraft` (limit pinjaman), lalu instansiasi objek `RekeningGiro`.
- Simpan objek tersebut ke dalam `daftarRekening` (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Bagian untuk kode program tugas 1 sebagai berikut.

```

        System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP");
        System.out.print("Masukkan Pilihan Produk: ");
        int pilihanProduk = input.nextInt();
        input.nextLine(); // Membersihkan Buffer Enter

        if (pilihanProduk == 1) {
            System.out.print("Masukkan Suku Bunga (dalam persen): ");
            double sukuBunga = input.nextDouble();
            input.nextLine();

            akunAktif = new RekeningTabungan(no, nama, saldo, pinAwal, sukuBunga);
            System.out.println("Sukses: Rekening Tabungan berhasil dibuat!");
        } else if (pilihanProduk == 2) {
            System.out.print("Masukkan Batas Overdraft: ");
            double batasOverdraft = input.nextDouble();
            input.nextLine();

            akunAktif = new RekeningGiro(no, nama, saldo, pinAwal, batasOverdraft);
            System.out.println("Sukses: Rekening Giro berhasil dibuat!");
        } else if (pilihanProduk == 3) {
            double saldoBonus = saldo + 100000;

            System.out.println("Selamat Datang di Rekening VIP! Anda mendapat Bonus Saldo Rp 100.000.");
            akunAktif = new RekeningVIP(no, nama, saldoBonus, pinAwal);
            System.out.println("Sukses: Rekening VIP berhasil dibuat!");
        } else {
            System.out.println("Error: Pilihan produk tidak valid! Pembuatan rekening dibatalkan.");
            break;
        }
    }
}

```

Penjelasan kode program di atas dibagi menjadi beberapa bagian sebagai berikut.

1. Opsi Produk

Menampilkan pilihan produk seperti Tabungan Umum, Giro Bisnis dan Rekening VIP pada challenge. Lalu meminta user untuk memilih diantara 3 pilihan tersebut.

2. Kondisi IF (Pilihan 1)

Pilihan 1 yaitu Tabungan Umum, program akan meminta user untuk memasukkan suku bunga dalam bentuk persen dimana bunga dengan tipe double. Lalu membuat objek RekeningTabungan. Objek disimpan di akunAktif yang

bertipe Rekening, jadi di sinilah upcasting terjadi. Lalu mengeluarkan output "Sukses: Rekening Tabungan berhasil dibuat!".

3. Kondisi IF (Pilihan 2)

Pilihan 2 yaitu Giro Bisnis, program meminta user untuk memasukkan Batas OverDraft dan Batas Overdraft dalam bentuk double. lalu membuat objek RekeningGiro, juga dengan upcasting ke akunAktif. Lalu mengeluarkan output "Sukses: Rekening Giro berhasil dibuat!".

4. Kondisi IF (Pilihan 3)

Pilihan 3 yaitu Rekening VIP dimana setiap membuat rekening akan mendapatkan saldo tambahan sebanyak Rp100.000. Pilihan 3 merupakan bagian dari challenge

5. Kondisi Else

Merupakan kondisi dimana user salah dalam memasukkan input (selain 1,2 dan 3). Lalu program akan mengeluarkan ouput "Error: Pilihan produk tidak valid! Pembuatan rekening dibatalkan."

Berikut beberapa output dari tugas 1

```
==== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
0. Keluar  
Pilih Menu: 1  
Masukkan No Rekening: 387562  
Masukkan Nama Pemilik: Sofian  
Masukkan Saldo Awal: 1000000  
Masukkan PIN (6 digit): 221006  
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP  
Masukkan Pilihan Produk:
```

```
Pilih Menu: 1
Masukkan No Rekening: 387562
Masukkan Nama Pemilik: Sofian
Masukkan Saldo Awal: 1000000
Masukkan PIN (6 digit): 221006
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP
Masukkan Pilihan Produk: 1
Masukkan Suku Bunga (dalam persen): 10
Rekening atas nama Sofian berhasil dibuat dengan saldo Rp1000000.0
Sukses: Rekening Tabungan berhasil dibuat!
```

```
==== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 230134
Masukkan Nama Pemilik: Sofian Ar
Masukkan Saldo Awal: 1500000
Masukkan PIN (6 digit): 230139
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP
Masukkan Pilihan Produk: 2
Masukkan Batas Overdraft: 10000000
Rekening atas nama Sofian Ar berhasil dibuat dengan saldo Rp1500000.0
Sukses: Rekening Giro berhasil dibuat!
```

2. Simulasi Waktu Akhir Bulan: (60%)

Pada tugas 2, program diminta untuk menambahkan menu 7 pada class main yaitu menu Simulasi Akhir Bulan dengan ketentuan sebagai berikut.

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2. Di dalam case 7, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

Langkah pertama tugas 2 yaitu menambahkan menu ke 7 pada class main. Berikut kode program untuk penambahan menu pada tugas 2.

```

while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening
    Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi
    Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Cetak Mutasi
    (Riwayat)");
    System.out.println("7. Simulasi Akhir
    Bulan (Khusus Tabungan)");
    System.out.println("0. Keluar");
    System.out.print("Pilih Menu: ")

```

```

case 7:
    if (akunAktif == null) {
        System.out.println("Error: Anda
        belum membuka rekening");
    } else if (akunAktif instanceof
    RekeningTabungan) {
        // Downcasting: Rekening ->
        RekeningTabungan
        RekeningTabungan tabungan =
        (RekeningTabungan) akunAktif;
        tabungan.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur
        bunga akhir bulan hanya berlaku untuk Rekening
        Tabungan.");
    }
    break;

```

Pertama mengecek apakah ada `akunAktif` yang sedang digunakan, lalu memeriksa dengan `instanceof RekeningTabungan` dengan memeriksa tipe objek asli di dalam `akunAktif`. Variabelnya bertipe `Rekening`, tetapi isinya bisa `RekeningTabungan`, `RekeningGiro`, atau `RekeningVIP`. Selanjutnya downcasting dan memanggil method, `RekeningTabungan tabungan = (RekeningTabungan) akunAktif`; merupakan downcasting eksplisit dari `Rekening` ke `RekeningTabungan`. Aman karena sudah dicek `instanceof`. Method `tambahBungaAkhirBulan()` hanya ada di `RekeningTabungan`, sehingga tidak bisa dipanggil lewat `akunAktif` langsung. Jika bukan bagian dari `RekeningTabungan` akan menampilkan pesan error "Gagal:

Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.". Berikut contoh outputnya.

```
==== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
0. Keluar  
Pilih Menu: 1  
Masukkan No Rekening: 219301  
Masukkan Nama Pemilik: Sofian  
Masukkan Saldo Awal: 1000000  
Masukkan PIN (6 digit): 221006  
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP  
Masukkan Pilihan Produk: 1  
Masukkan Suku Bunga (dalam persen): 5  
Rekening atas nama Sofian berhasil dibuat dengan saldo Rp1000000.0  
Sukses: Rekening Tabungan berhasil dibuat!
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
0. Keluar  
Pilih Menu: 7  
Bunga 5.0% berhasil ditambahkan Rp50000.0
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
0. Keluar  
Pilih Menu: 4  
-- INFO REKENING --  
No. Rekening : 219301  
Nama Pemilik : Sofian  
Saldo Akhir : Rp1050000.0  
Status Akun : Aktif  
-----
```

3. Link Repository

Berikut dibawah ini merupakan link github untuk laporan praktikum pekan 4
[sofianarbai/PrakPBO_C_2511533029](https://github.com/sofianarbai/PrakPBO_C_2511533029) at [Praktikum-PBO-Pekan-4](#)